ASSIGNMENT 8

REAL ESTATE MARKET DATA WRANGLING

Detailed code walkthrough • Core theory • Viva and cross-question bank

IN THIS GUIDE	YOU WILL BE ABLE TO
Explain every notebook cell and the reason for each operation.	Describe what each function returns and how it affects the data.
Explain cleaning, filtering, grouping, outliers and encoding.	Answer basic theory and deeper examiner follow-ups.
Identify limitations and small improvements in the submitted code.	Explain the practical confidently in simple language.

Experiment: Data Wrangling on Real Estate Market. The uploaded lab schedule identifies the dataset as *RealEstate_Prices.csv*. The notebook loads, cleans, filters, groups, detects outliers, encodes a category and exports the processed data.

Prepared from the uploaded Assignment_8.ipynb and the Computer Laboratory I assignment schedule. Explanations use beginner-friendly language.

1. Aim, dataset and complete workflow

Aim: convert a raw real-estate CSV into a cleaner, more consistent dataset and perform a small analysis of house prices. Raw data can contain unclear column names, missing values, duplicate rows and inconsistent representations. Data wrangling prepares it for analysis.

Dataset fields after renaming:

Column	Meaning
id	Record identifier.
transaction_date	Transaction date-related field in the source dataset.
house_age	Age of the house.
distance_to_mrt	Distance to the nearest MRT station.
stores	Number of nearby convenience stores.
latitude / longitude	Geographical coordinates.
price_per_unit	House price per unit area; the value being summarised.

Workflow: CSV load → inspect → standardise column labels → check missing values and duplicates → clean rows and fill numeric gaps → filter stores ≥ 5 → create age categories → group and calculate price statistics → flag outliers → one-hot encode age category → export CSV.

Important: This notebook is not training a machine-learning model. It is data preparation and exploratory analysis. The schedule (pages 3–4) also mentions merging additional relevant datasets, but no merge operation appears in the submitted notebook.

Key terms

Data wrangling: cleaning and transforming raw data into a usable structure. **Feature:** an input attribute describing a record. **Target:** the outcome of interest; here price_per_unit is the outcome being analysed. **Exploratory analysis:** summarising and inspecting data to understand patterns.

2. Imports, loading and inspection

Cell 0 – import libraries

```
import pandas as pd
import numpy as np
from pathlib import Path
```

Import	Purpose in this notebook
pandas as pd	Provides DataFrames and functions such as read_csv(), isna(), groupby() and to_csv().
numpy as np	Numerical computing library. Imported here, although no NumPy function is explicitly used in the shown cells.
Path	Path and folder utilities from Python's pathlib module. Used to create the output folder.

An alias such as pd is simply a shorter name. It does not change the library.

Cell 1 – read the CSV

```
homes = pd.read_csv('RealEstate_Prices.csv')
```

read_csv() reads a comma-separated file and returns a pandas DataFrame. A DataFrame is a two-dimensional labelled table with rows and columns. The result is assigned to homes. The file must be in the current working directory, otherwise Python raises a file-not-found error.

Cell 2 – preview records

```
homes.head()
```

head() displays the first five rows by default. You can use homes.head(10) to see ten rows. It helps check whether loading worked and whether the columns and values look sensible.

Useful inspection commands for viva

```
homes.shape # number of rows and columns
homes.info() # column names, data types, non-null counts
homes.describe() # summary statistics of numeric columns
homes.tail() # last five rows
```

Cross-question: What is a Series? A Series is a one-dimensional labelled data structure. Selecting one DataFrame column, for example homes['stores'], normally returns a Series.

3. Standardising column names

Cell 3 – clean and rename labels

```
homes.columns = homes.columns.str.strip().str.lower().str.replace(
r'[^a-z0-9]+', '_', regex=True)

homes = homes.rename(columns={
'no': 'id',
'x1_transaction_date': 'transaction_date',
'x2_house_age': 'house_age',
'x3_distance_to_the_nearest_mrt_station': 'distance_to_mrt',
'x4_number_of_convenience_stores': 'stores',
'x5_latitude': 'latitude',
'x6_longitude': 'longitude',
'y_house_price_of_unit_area': 'price_per_unit'
})
```

Part	Meaning
homes.columns	Accesses the column labels.
.str	Applies string operations to every label.
strip()	Removes leading and trailing whitespace.
lower()	Converts labels to lowercase.
replace(..., regex=True)	Replaces runs of characters outside a–z and 0–9 with underscores.
rename(columns={...})	Maps selected old names to clearer new names.

Regex explained: r makes a raw string. [ ] defines a character set. ^ at the start means NOT. a-z0-9 means lowercase letters or digits. + means one or more consecutive matches. So spaces and punctuation become underscores.

Why do this? Clean names are easier to type and reduce mistakes. For example, distance_to_mrt is easier to use than a long source label.

Cell 4 – inspect again

```
homes.head()
```

This confirms the data remains present and the new column names are visible. Renaming columns does not alter the row values.

Cross-question: Does rename() modify homes automatically? By default, it returns a new DataFrame. Assigning it back to homes stores the renamed result.

4. Missing values, duplicates and imputation

Cell 5 – count missing values

```
homes.isna().sum()
```

isna() creates a True/False result for every cell: True means missing. sum() counts True values column-wise (True behaves like 1, False like 0). It shows how many missing values each column has.

Cell 6 – count duplicate rows

```
homes.duplicated().sum()
```

duplicated() marks rows that repeat an earlier row; by default it compares all columns. sum() counts the duplicate flags. It only detects duplicates; it does not remove them.

Cell 7 – count rows

```
len(homes)
```

len(DataFrame) returns the number of rows. It is useful to compare dataset size before and after cleaning.

Cell 8 – remove duplicates and missing target prices

```
homes = homes.drop_duplicates().dropna(subset='price_per_unit')
```

drop_duplicates() removes repeated rows. **dropna(subset='price_per_unit')** removes rows where the price is missing. The subset argument means only the target-price column is checked for missingness.

Cell 9 – fill missing feature values

```
features = homes.columns.difference(['id', 'price_per_unit'])
homes[features] = homes[features].fillna(homes[features].median())
print('Cleaned records:', len(homes))
```

columns.difference() selects all column names except id and price_per_unit. median() calculates a median for each selected numeric column. fillna() fills missing values with the corresponding column median. The print statement reports the remaining row count.

Why median? The median is the middle sorted value and is less affected by extreme values than the mean. It is often useful for skewed numeric data. Limitation: this assumes the feature columns are numeric. Categorical missing values need a strategy such as mode or an 'Unknown' label. The target price is dropped when missing, not imputed.

5. Filtering, age bins and aggregation

Cell 10 – filter properties

```
selected = homes[homes['stores'] >= 5].copy()
selected.head()
```

homes['stores'] >= 5 creates a Boolean Series. homes[condition] keeps only rows where the condition is True. copy() creates an independent copy so later changes to selected do not unintentionally affect homes.

Cell 11 – count selected rows

```
len(selected)
```

Returns the number of properties with at least five nearby convenience stores. This count cannot be greater than the cleaned row count.

Cell 12 – bin house age and summarise prices

```
selected['age_group'] = pd.cut(
selected['house_age'], bins=[-1, 10, 30, 100],
labels=['up to 10', '10 to 30', 'over 30'])

summ = selected.groupby(['age_group', 'stores'], observed=True)
['price_per_unit'].agg(['count', 'mean', 'median'])
display(summ)
```

pd.cut() converts a continuous numeric column into categories. The bin edges create three intervals. By default, intervals are right-closed: (–1,10], (10,30], (30,100]. So age exactly 10 is in the first group; age exactly 30 is in the second.

groupby() splits rows by the combination of age_group and stores. Selecting price_per_unit chooses the column to summarise. **agg()** applies multiple calculations: count (number of valid prices), mean (average), median (middle value). **observed=True** shows category combinations that actually occur. display() renders the table in Jupyter.

Statistic	Meaning	Remember
count	Number of valid price values in a group.	Shows group size.
mean	Sum divided by number of values.	Affected by extreme prices.
median	Middle value after sorting.	More robust to outliers.

Potential issue: Values outside the supplied age range become missing in age_group. pd.cut() does not predict or modify the age; it only assigns category labels.

6. IQR outliers, encoding and export

Cell 13 – flag potential outliers

```
q1 = selected['price_per_unit'].quantile(0.25)
q3 = selected['price_per_unit'].quantile(0.75)
iqr = q3 - q1
upper = q3 + 1.5 * iqr
lower = q1 - 1.5 * iqr
outliers = ~selected['price_per_unit'].between(lower, upper)
print(outliers.sum())
```

Q1 is the 25th percentile and Q3 is the 75th percentile. IQR = Q3 – Q1 and describes the spread of the middle 50% of values. The standard fences are Q1 – 1.5×IQR and Q3 + 1.5×IQR. Values outside the fences are flagged as potential outliers.

between(lower, upper) is True for values inside the range (inclusive by default). The ~ symbol reverses the Boolean values, so outliers is True for values outside the range. sum() counts them.

Important: the code detects and counts outliers only. It does not delete them. An unusually high property price may be genuine, not a data error.

Cell 14 – one-hot encode and export

```
encoded = pd.get_dummies(selected, columns=['age_group'], dtype=int)
Path('outputs').mkdir(exist_ok=True)
encoded.to_csv('output.csv', index=False)
display(encoded.head())
```

get_dummies() converts a category column into binary indicator columns. For example, the three age groups become columns such as age_group_up to 10, age_group_10 to 30 and age_group_over 30. Each row has 1 for its category and 0 for the others. dtype=int requests integer values.

Path(...).mkdir(exist_ok=True) creates the outputs folder; exist_ok=True avoids an error if it already exists. **to_csv(..., index=False)** exports the table without adding the pandas row index as an extra column.

Small but important detail: the code creates outputs, but saves to output.csv in the current directory. To save inside the folder, use:

```
encoded.to_csv(Path('outputs') / 'output.csv', index=False)
```

get_dummies() returns a transformed DataFrame. It does not overwrite selected because the result is assigned to encoded.

7. Theory concepts for the viva

Concept	Simple definition
Data cleaning	Finding and handling missing, duplicate, invalid or inconsistent data.
DataFrame	Two-dimensional labelled table in pandas.
Series	One-dimensional labelled array, often returned by selecting one column.
Missing value	Absent or unknown value, often represented by NaN.
Imputation	Replacing missing values using mean, median, mode or a fixed label/value.
Duplicate record	Repeated row/entity that may distort analysis.
Filtering	Keeping rows that satisfy a condition.
Binning / discretisation	Converting numeric values into ranges or categories.
Aggregation	Calculating group-level summaries such as count, mean or median.
Outlier	Value unusually far from most observations; may still be valid.
IQR	Q3 − Q1; spread of the middle half of the data.
Categorical variable	A variable representing a label or group.
One-hot encoding	Representing each category using a separate 0/1 column.
CSV	Plain-text tabular file format with comma-separated fields.
Feature scaling	Changing numeric feature ranges, e.g. min-max scaling or standardisation.
Data leakage	Using information in preparation/training that would not legitimately be available at prediction time.

Related concepts likely to be asked

Mean vs median: mean is sensitive to extreme values; median is more robust. **Mode:** most frequent value, often used for categorical imputation. **Drop vs fill:** dropping loses rows; filling retains rows but inserts an estimate. **Encoding:** converts categories into numeric form. **Scaling:** changes numeric feature ranges; this notebook does not scale. **Merge:** joins DataFrames using common keys. **Concat:** stacks DataFrames along rows or columns.

Why prepare data before machine learning?

Many algorithms need consistent data types and complete numeric inputs. Missing values, duplicates and inconsistent formats can cause errors or misleading patterns. Good preparation improves the reliability of later analysis.

8. Viva questions and model answers

1. What is the aim of this practical?
To clean real-estate data, filter relevant properties and summarise house prices by house-age group and nearby store count.

2. Is this a machine-learning model?
No. It is data wrangling and exploratory analysis. A model could be trained later using price_per_unit as a target.

3. What does read_csv() return?
A pandas DataFrame.

4. Difference between isna() and dropna()?
isna detects missing values; dropna removes rows or columns containing missing values.

5. Difference between duplicated() and drop_duplicates()?
duplicated flags repeated rows; drop_duplicates removes them.

6. Why not fill missing price_per_unit with median?
It is the outcome being analysed. The notebook removes rows without a known target price.

7. Why median imputation?
Median is less sensitive to extreme values than mean. It is applied separately to each numeric feature column.

8. What does stores >= 5 return?
A Boolean Series containing True or False for each row.

9. Why use copy()?
To make a separate DataFrame for subsequent changes and avoid chained-assignment/view issues.

10. What is pd.cut() used for?
To convert numeric house ages into interval categories.

11. What does groupby() do?
It divides records into groups based on one or more columns so group-level calculations can be performed.

12. What does agg() do?
It applies one or more aggregation functions, here count, mean and median.

13. What is IQR?
Q3 minus Q1, the range covered by the middle 50% of observations.

14. Does this code remove outliers?
No. It flags and counts them only.

15. Are outliers always incorrect?
No. They can be genuine unusual properties. They should be investigated before removal.

16. Why use one-hot encoding?
To represent categories as numeric 0/1 columns, which many machine-learning algorithms can use.

17. Why index=False?
To avoid exporting the DataFrame's row index as an extra CSV column.

18. What is the difference between merge() and concat()?
merge joins rows using matching keys; concat stacks tables along an axis.

19. Does the notebook merge datasets?
No merge operation is shown, even though the lab schedule mentions merging if relevant data is available.

20. What is the dummy-variable trap?
In some linear models, keeping every one-hot column along with an intercept creates perfect multicollinearity. drop_first=True can avoid this when appropriate.

9. Tricky cross-questions, improvements and speaking answer

21. What happens if house_age is outside −1 to 100?
pd.cut() assigns a missing category because the value is outside the bin edges.

22. What happens if features contains a text column?
Median is not meaningful for text. Use a categorical strategy such as mode or an 'Unknown' category.

23. Why is observed=True used?
It asks groupby to include only category combinations that are actually present in the data.

24. What does ~ mean in Python?
For a Boolean Series, it performs element-wise logical NOT.

25. What is the difference between fit and transform?
In machine learning, fit learns parameters from data; transform applies a learned transformation. This notebook does not use a scikit-learn transformer.

26. How could two datasets be joined?
For example, pd.merge(left, right, on='id', how='inner'). The key and join type depend on the data and desired records.

27. How could you inspect the actual outlier rows?
Use selected.loc[outliers] to display records whose price is outside the IQR fences.

28. How would you count missing values after cleaning?
Use homes.isna().sum() again and compare with the initial counts.

Improvements you can suggest

- Check data types using homes.info() before median imputation.
- Inspect missing values and age ranges after pd.cut().
- Display selected.loc[outliers] before deciding what to do with unusual prices.
- Save output inside outputs using Path('outputs') / 'output.csv'.
- Report row counts before and after every cleaning operation.
- If modelling later, learn imputation/scaling values from training data only to avoid data leakage.

One-minute explanation to speak

“First, I imported pandas, NumPy and Path, and loaded the real-estate CSV into a DataFrame. I inspected the first rows, cleaned the column names and renamed them to meaningful labels. I checked missing values and duplicates, removed duplicate rows and records without a target price, and filled missing numeric feature values using each column's median. Then I selected properties with at least five convenience stores. I divided house ages into three categories using pd.cut and grouped the data by age group and store count to calculate count, mean and median prices. I flagged potential price outliers using the 1.5-times-IQR rule. Finally, I one-hot encoded the age categories and exported the processed DataFrame to CSV.”

Final revision checklist: DataFrame vs Series • isna vs dropna • duplicated vs drop_duplicates • mean vs median • filtering vs groupby • pd.cut intervals • IQR fences • outlier vs error • one-hot encoding • merge vs concat • index=False • output folder path.